

Examen final de Programación 1

Angel Estuardo Campos Santay

Codigo fuente

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <iomanip>
#include <cstring>
#include <cmath>
#include <algorithm>

using namespace std;

struct Producto
{
    int codigo;
    char nombre[50];
    double precio;
    int stock;
    int vendidos;
    bool activo;
};

struct DetalleVenta
{
    int codigoProducto;
    char nombreProducto[50];
    int cantidad;
    double precioUnitario;
    double subtotal;
};

struct Venta
{
    int idVenta;
    char fecha[15];
    int cantidadArticulos;
    double subtotal;
    double iva;
    double descuento;
    double total;
};

double ventasPorMes[12][31] = {0};

void registrarProducto();
void listarProductos();
void buscarProductoCodigo();
void buscarProductoNombre();
void actualizarStock();
void modificarPrecio();
void eliminarProducto();

void menuProductos();

void realizarVenta();

void productosMenorStock();
void productosMasVendidos();
void ventasDelDia();
void reporteVentasMes();

void ordenarPrecioAsc();
void ordenarPrecioDesc();
void ordenarStockAsc();
void ordenarStockDesc();
void ordenarVendidos();

void exportarReporteTXT();
void reiniciarInventario();
void estadisticasGenerales();

int generarIdVenta();

int main()
{
    int opcion;

    do
    {
        cout << "\n===== \n";
        cout << "Examen Final Programacion \n";
        cout << "Por Angel Estuardo Campos Santay \n";
        cout << "9941-25-4809 \n";
        cout << "===== \n";

        cout << "\nMENU PRINCIPAL \n";
        cout << "1. Gestion de Productos \n";
        cout << "2. Realizar Venta \n";
        cout << "3. Reportes \n";
```

```

cout << "4. Utilidades\n";
cout << "5. Salir\n";
cout << "\nSeleccione una opcion: ";

cin >> opcion;

switch(opcion)
{
    case 1:
        menuProductos();
        break;

    case 2:
        realizarVenta();
        break;

    case 3:
    {
        int opReporte;

        do
        {
            cout << "\n===== REPORTES =====\n";
            cout << "1. Productos con menor stock\n";
            cout << "2. Productos mas vendidos\n";
            cout << "3. Ventas del dia\n";
            cout << "4. Ventas por mes\n";
            cout << "5. Ordenar precio ascendente\n";
            cout << "6. Ordenar precio descendente\n";
            cout << "7. Ordenar stock ascendente\n";
            cout << "8. Ordenar stock descendente\n";
            cout << "9. Ordenar por ventas acumuladas\n";
            cout << "0. Regresar\n";
            cout << "Opcion: ";

            cin >> opReporte;

            switch(opReporte)
            {
                case 1:
                    productosMenorStock();
                    break;

                case 2:
                    productosMasVendidos();
                    break;

                case 3:
                    ventasDelDia();
                    break;

                case 4:
                    reporteVentasMes();
                    break;

                case 5:
                    ordenarPrecioAsc();
                    break;

                case 6:
                    ordenarPrecioDesc();
                    break;

                case 7:
                    ordenarStockAsc();
                    break;

                case 8:
                    ordenarStockDesc();
                    break;

                case 9:
                    ordenarVendidos();
                    break;
            }

        } while(opReporte != 0);

        break;
    }

    case 4:
    {
        int opUtilidad;

        do
        {
            cout << "\n===== UTILIDADES =====\n";
            cout << "1. Exportar reporte TXT\n";
            cout << "2. Reiniciar inventario\n";
            cout << "3. Estadísticas generales\n";
            cout << "0. Regresar\n";
            cout << "Opcion: ";

            cin >> opUtilidad;

            switch(opUtilidad)
            {

```

```

        case 1:
            exportarReporteTXT();
            break;

        case 2:
            reiniciarInventario();
            break;

        case 3:
            estadisticasGenerales();
            break;
    }

    } while(opUtilidad != 0);

    break;
}

case 5:
    cout << "\nPrograma finalizado.\n";
    break;

default:
    cout << "\nOpcion invalida.\n";
}

} while(opcion != 5);

return 0;
}

void menuProductos()
{
    int opcion;

    do
    {
        cout << "\n===== GESTION DE PRODUCTOS =====\n";
        cout << "1. Registrar Producto\n";
        cout << "2. Listar Productos\n";
        cout << "3. Buscar porCodigo\n";
        cout << "4. Buscar por Nombre\n";
        cout << "5. Actualizar Stock\n";
        cout << "6. Modificar Precio\n";
        cout << "7. Eliminar/Desactivar Producto\n";
        cout << "0. Regresar\n";
        cout << "Opcion: ";

        cin >> opcion;

        switch(opcion)
        {
            case 1:
                registrarProducto();
                break;

            case 2:
                listarProductos();
                break;

            case 3:
                buscarProductoCodigo();
                break;

            case 4:
                buscarProductoNombre();
                break;

            case 5:
                actualizarStock();
                break;

            case 6:
                modificarPrecio();
                break;

            case 7:
                eliminarProducto();
                break;
        }

    } while(opcion != 0);
}

void registrarProducto()
{
    try
    {
        Producto p;

        cout << "\nCodigo: ";
        cin >> p.codigo;

        cin.ignore();

        cout << "Nombre: ";
        cin.getline(p.nombre,50);

        cout << "Precio: ";
    }
}

```

```

    cin >> p.precio;

    cout << "Stock: ";
    cin >> p.stock;

    if(p.precio <= 0 || p.stock < 0)
    {
        throw string("Datos invalidos.");
    }

    p.vendidos = 0;
    p.activo = true;

    ofstream archivo("productos.dat", ios::binary | ios::app);

    if(!archivo)
    {
        throw string("No se pudo abrir productos.dat");
    }

    archivo.write((char*)&p, sizeof(Producto));
    archivo.close();

    cout << "\nProducto registrado correctamente.\n";
}
catch(string mensaje)
{
    cout << "\nError: " << mensaje << endl;
}
}

void listarProductos()
{
    Producto p;

    ifstream archivo("productos.dat", ios::binary);

    if(!archivo)
    {
        cout << "\nNo existe archivo de productos.\n";
        return;
    }

    cout << "\n===== \n";

    while(archivo.read((char*)&p, sizeof(Producto)))
    {
        if(p.activo)
        {
            cout << "\nCodigo: " << p.codigo;
            cout << "\nNombre: " << p.nombre;
            cout << "\nPrecio: Q" << fixed << setprecision(2) << p.precio;
            cout << "\nStock: " << p.stock;
            cout << "\nVendidos: " << p.vendidos;
            cout << "\n-----";
        }
    }

    archivo.close();
}

void buscarProductoCodigo()
{
    int codigo;
    bool encontrado = false;

    cout << "\ningrese codigo: ";
    cin >> codigo;

    Producto p;

    ifstream archivo("productos.dat", ios::binary);

    while(archivo.read((char*)&p, sizeof(Producto)))
    {
        if(p.codigo == codigo && p.activo)
        {
            encontrado = true;

            cout << "\nCodigo: " << p.codigo;
            cout << "\nNombre: " << p.nombre;
            cout << "\nPrecio: Q" << p.precio;
            cout << "\nStock: " << p.stock;
            cout << "\nVendidos: " << p.vendidos;
            break;
        }
    }

    archivo.close();

    if(!encontrado)
    {
        cout << "\nProducto no encontrado.\n";
    }
}

void buscarProductoNombre()
{

```

```

char nombreBuscado[50];
bool encontrado = false;

cin.ignore();

cout << "\nNombre a buscar: ";
cin.getline(nombreBuscado,50);

Producto p;

ifstream archivo("productos.dat", ios::binary);

while(archivo.read((char*)&p,sizeof(Producto)))
{
    if(strstr(p.nombre,nombreBuscado) != NULL && p.activo)
    {
        encontrado = true;

        cout << "\nCodigo: " << p.codigo;
        cout << "\nNombre: " << p.nombre;
        cout << "\nPrecio: Q" << p.precio;
        cout << "\nStock: " << p.stock;
        cout << "\nVendidos: " << p.vendidos;
        cout << "\n-----";
    }
}

archivo.close();

if(!encontrado)
{
    cout << "\nNo se encontraron coincidencias.\n";
}
}

void actualizarStock()
{
    int codigo;
    int nuevoStock;

    cout << "\nCodigo producto: ";
    cin >> codigo;

    cout << "Nuevo stock: ";
    cin >> nuevoStock;

    fstream archivo("productos.dat", ios::binary | ios::in | ios::out);

    Producto p;
    bool encontrado = false;

    while(archivo.read((char*)&p,sizeof(Producto)))
    {
        if(p.codigo == codigo && p.activo)
        {
            encontrado = true;

            p.stock = nuevoStock;

            archivo.seekp(-sizeof(Producto),ios::cur);
            archivo.write((char*)&p,sizeof(Producto));

            break;
        }
    }

    archivo.close();

    if(encontrado)
        cout << "\nStock actualizado.\n";
    else
        cout << "\nProducto no encontrado.\n";
}

void modificarPrecio()
{
    int codigo;
    double nuevoPrecio;

    cout << "\nCodigo producto: ";
    cin >> codigo;

    cout << "Nuevo precio: ";
    cin >> nuevoPrecio;

    fstream archivo("productos.dat", ios::binary | ios::in | ios::out);

    Producto p;
    bool encontrado = false;

    while(archivo.read((char*)&p,sizeof(Producto)))
    {
        if(p.codigo == codigo && p.activo)
        {
            encontrado = true;

            p.precio = nuevoPrecio;

```

```

        archivo.seekp(-sizeof(Producto),ios::cur);
        archivo.write((char*)&p,sizeof(Producto));

        break;
    }
}

archivo.close();

if(encontrado)
    cout << "\nPrecio actualizado.\n";
else
    cout << "\nProducto no encontrado.\n";
}

void eliminarProducto()
{
    int codigo;

    cout << "\nCodigo producto: ";
    cin >> codigo;

    fstream archivo("productos.dat", ios::binary | ios::in | ios::out);

    Producto p;
    bool encontrado = false;

    while(archivo.read((char*)&p,sizeof(Producto)))
    {
        if(p.codigo == codigo)
        {
            encontrado = true;

            p.activo = false;

            archivo.seekp(-sizeof(Producto),ios::cur);
            archivo.write((char*)&p,sizeof(Producto));

            break;
        }
    }

    archivo.close();

    if(encontrado)
        cout << "\nProducto desactivado.\n";
    else
        cout << "\nProducto no encontrado.\n";
}

int generarIdVenta()
{
    Venta v;
    int ultimo = 0;

    ifstream archivo("ventas.dat", ios::binary);

    while(archivo.read((char*)&v,sizeof(Venta)))
    {
        ultimo = v.idVenta;
    }

    archivo.close();

    return ultimo + 1;
}

void realizarVenta()
{
    try
    {
        vector<DetalleVenta> detalles;

        int codigo;
        int cantidad;
        int totalArticulos = 0;

        double subtotalGeneral = 0;
        double descuento = 0;
        double iva = 0;
        double total = 0;

        while(true)
        {
            cout << "\nCodigo producto (0 para finalizar): ";
            cin >> codigo;

            if(codigo == 0)
                break;

            cout << "Cantidad: ";
            cin >> cantidad;

            if(cantidad <= 0)
            {
                throw string("Cantidad invalida.");
            }

            fstream archivo("productos.dat", ios::binary | ios::in | ios::out);

```

```

Producto p;
bool encontrado = false;

while(archivo.read((char*)&p,sizeof(Producto)))
{
    if(p.codigo == codigo && p.activo)
    {
        encontrado = true;

        if(cantidad > p.stock)
        {
            cout << "\nStock insuficiente.\n";
            archivo.close();
            break;
        }

        DetalleVenta d;

        d.codigoProducto = p.codigo;

        strcpy(d.nombreProducto,p.nombre);

        d.cantidad = cantidad;

        d.precioUnitario = p.precio;

        d.subtotal = cantidad * p.precio;

        subtotalGeneral += d.subtotal;

        totalArticulos += cantidad;

        detalles.push_back(d);

        p.stock -= cantidad;
        p.vendidos += cantidad;

        archivo.seekp(-sizeof(Producto),ios::cur);
        archivo.write((char*)&p,sizeof(Producto));

        cout << "\nProducto agregado.\n";

        break;
    }
}

archivo.close();

if(!encontrado)
{
    cout << "\nProducto no encontrado.\n";
}

if(detalles.empty())
{
    cout << "\nNo se registraron productos.\n";
    return;
}

if(subtotalGeneral > 500)
{
    descuento += subtotalGeneral * 0.05;
}

if(totalArticulos % 5 == 0)
{
    descuento += 25;
}

iva = subtotalGeneral * 0.12;

total = subtotalGeneral + iva - descuento;

total = round(total * 100) / 100.0;

Venta venta;

venta.idVenta = generarIdVenta();

cout << "\nIngrese fecha (dd/mm/yyyy): ";
cin >> venta.fecha;

venta.cantidadArticulos = totalArticulos;
venta.subtotal = subtotalGeneral;
venta.iva = iva;
venta.descuento = descuento;
venta.total = total;

ofstream guardar("ventas.dat", ios::binary | ios::app);

guardar.write((char*)&venta,sizeof(Venta));

guardar.close();

int dia;
int mes;

```

```

int anio;

sscanf(venta.fecha,"%d/%d/%d",&dia,&mes,&anio);

if(mes >= 1 && mes <= 12 && dia >= 1 && dia <= 31)
{
    ventasPorMes[mes - 1][dia - 1] += total;
}

cout << "\n===== ";
cout << "\nID Venta: " << venta.idVenta;
cout << "\nSubtotal: Q" << fixed << setprecision(2) << subtotalGeneral;
cout << "\nIVA: Q" << iva;
cout << "\nDescuento: Q" << descuento;
cout << "\nTotal: Q" << total;
cout << "\n===== \n";
}
catch(string mensaje)
{
    cout << "\nError: " << mensaje << endl;
}
}

void productosMenorStock()
{
    Producto p;

    ifstream archivo("productos.dat", ios::binary);

    if(!archivo)
    {
        cout << "\nNo existe archivo.\n";
        return;
    }

    cout << "\n===== PRODUCTOS CON MENOR STOCK =====\n";

    while(archivo.read((char*)&p,sizeof(Producto)))
    {
        if(p.activo && p.stock <= 5)
        {
            cout << "\nCodigo: " << p.codigo;
            cout << "\nNombre: " << p.nombre;
            cout << "\nStock: " << p.stock;
            cout << "\n-----";
        }
    }

    archivo.close();
}

void productosMasVendidos()
{
    vector<Producto> productos;

    Producto p;

    ifstream archivo("productos.dat", ios::binary);

    while(archivo.read((char*)&p,sizeof(Producto)))
    {
        if(p.activo)
        {
            productos.push_back(p);
        }
    }

    archivo.close();

    for(size_t i = 0; i < productos.size(); i++)
    {
        for(size_t j = i + 1; j < productos.size(); j++)
        {
            if(productos[j].vendidos > productos[i].vendidos)
            {
                swap(productos[i], productos[j]);
            }
        }
    }

    cout << "\n===== PRODUCTOS MAS VENDIDOS =====\n";

    for(size_t i = 0; i < productos.size(); i++)
    {
        cout << "\nCodigo: " << productos[i].codigo;
        cout << "\nNombre: " << productos[i].nombre;
        cout << "\nVendidos: " << productos[i].vendidos;
        cout << "\n-----";
    }
}

void ventasDelDia()
{
    Venta v;

    char fechaBuscada[15];

    cout << "\ningrese fecha (dd/mm/yyyy): ";
    cin >> fechaBuscada;
}

```



```

double totalDia = 0;

ifstream archivo("ventas.dat", ios::binary);

while(archivo.read((char*)&v,sizeof(Venta)))
{
    if(strcmp(v.fecha, fechaBuscada) == 0)
    {
        totalDia += v.total;
    }
}

archivo.close();

cout << "\nVentas del dia: Q";
cout << fixed << setprecision(2) << totalDia << endl;
}

void reporteVentasMes()
{
    int mes;

    cout << "\nIngrese mes (1-12): ";
    cin >> mes;

    if(mes < 1 || mes > 12)
    {
        cout << "\nMes invalido.\n";
        return;
    }

    double totalMes = 0;

    cout << "\n===== REPORTE DEL MES =====\n";

    for(int dia = 0; dia < 31; dia++)
    {
        if(ventasPorMes[mes - 1][dia] > 0)
        {
            cout << "Dia "
                << dia + 1
                << ": Q"
                << fixed
                << setprecision(2)
                << ventasPorMes[mes - 1][dia]
                << endl;

            totalMes += ventasPorMes[mes - 1][dia];
        }
    }

    cout << "\nTotal Mes: Q";
    cout << fixed << setprecision(2) << totalMes << endl;
}

void ordenarPrecioAsc()
{
    vector<Producto> productos;

    Producto p;

    ifstream archivo("productos.dat", ios::binary);

    while(archivo.read((char*)&p,sizeof(Producto)))
    {
        if(p.activo)
        {
            productos.push_back(p);
        }
    }

    archivo.close();

    for(size_t i = 0; i < productos.size() - 1; i++)
    {
        for(size_t j = 0; j < productos.size() - i - 1; j++)
        {
            if(productos[j].precio > productos[j + 1].precio)
            {
                swap(productos[j], productos[j + 1]);
            }
        }
    }

    cout << "\n===== PRECIO ASCENDENTE =====\n";

    for(auto p : productos)
    {
        cout << p.codigo
            << " - "
            << p.nombre
            << " - Q"
            << p.precio
            << endl;
    }
}

```

```

void ordenarPrecioDesc()
{
    vector<Producto> productos;

    Producto p;

    ifstream archivo("productos.dat", ios::binary);

    while(archivo.read((char*)&p,sizeof(Producto)))
    {
        if(p.activo)
        {
            productos.push_back(p);
        }
    }

    archivo.close();

    for(size_t i = 0; i < productos.size() - 1; i++)
    {
        for(size_t j = 0; j < productos.size() - i - 1; j++)
        {
            if(productos[j].precio < productos[j + 1].precio)
            {
                swap(productos[j], productos[j + 1]);
            }
        }
    }

    cout << "\n===== PRECIO DESCENDENTE =====\n";

    for(auto p : productos)
    {
        cout << p.codigo
              << " - "
              << p.nombre
              << " - Q"
              << p.precio
              << endl;
    }
}

void ordenarStockAsc()
{
    vector<Producto> productos;

    Producto p;

    ifstream archivo("productos.dat", ios::binary);

    while(archivo.read((char*)&p,sizeof(Producto)))
    {
        if(p.activo)
        {
            productos.push_back(p);
        }
    }

    archivo.close();

    for(size_t i = 0; i < productos.size() - 1; i++)
    {
        int min = i;

        for(size_t j = i + 1; j < productos.size(); j++)
        {
            if(productos[j].stock < productos[min].stock)
            {
                min = j;
            }
        }

        swap(productos[i], productos[min]);
    }

    cout << "\n===== STOCK ASCENDENTE =====\n";

    for(auto p : productos)
    {
        cout << p.codigo
              << " - "
              << p.nombre
              << " - "
              << p.stock
              << endl;
    }
}

void ordenarStockDesc()
{
    vector<Producto> productos;

    Producto p;

    ifstream archivo("productos.dat", ios::binary);

    while(archivo.read((char*)&p,sizeof(Producto)))
    {

```

```

        if(p.activo)
        {
            productos.push_back(p);
        }
    }

    archivo.close();

    for(size_t i = 0; i < productos.size() - 1; i++)
    {
        int max = i;

        for(size_t j = i + 1; j < productos.size(); j++)
        {
            if(productos[j].stock > productos[max].stock)
            {
                max = j;
            }
        }

        swap(productos[i], productos[max]);
    }

    cout << "\n===== STOCK DESCENDENTE =====\n";

    for(auto p : productos)
    {
        cout << p.codigo
              << " - "
              << p.nombre
              << " - "
              << p.stock
              << endl;
    }
}

void ordenarVendidos()
{
    vector<Producto> productos;

    Producto p;

    ifstream archivo("productos.dat", ios::binary);

    while(archivo.read((char*)&p, sizeof(Producto)))
    {
        if(p.activo)
        {
            productos.push_back(p);
        }
    }

    archivo.close();

    for(size_t i = 0; i < productos.size(); i++)
    {
        for(size_t j = i + 1; j < productos.size(); j++)
        {
            if(productos[j].vendidos > productos[i].vendidos)
            {
                swap(productos[i], productos[j]);
            }
        }
    }

    cout << "\n===== VENTAS ACUMULADAS =====\n";

    for(auto p : productos)
    {
        cout << p.codigo
              << " - "
              << p.nombre
              << " - "
              << p.vendidos
              << endl;
    }
}

void exportarReporteTXT()
{
    ofstream reporte("reporte.txt");

    if(!reporte)
    {
        cout << "\nNo se pudo crear el reporte.\n";
        return;
    }

    Producto p;

    ifstream archivo("productos.dat", ios::binary);

    reporte << "===== REPORTE GENERAL DE PRODUCTOS =====\n\n";

    while(archivo.read((char*)&p, sizeof(Producto)))
    {
        if(p.activo)
        {
            reporte << "Codigo: " << p.codigo << endl;
        }
    }
}

```

```

        reporte << "Nombre: " << p.nombre << endl;
        reporte << "Precio: Q" << fixed << setprecision(2) << p.precio << endl;
        reporte << "Stock: " << p.stock << endl;
        reporte << "Vendidos: " << p.vendidos << endl;
        reporte << "-----" << endl;
    }
}

archivo.close();
reporte.close();

cout << "\nReporte exportado a reporte.txt\n";
}

void reiniciarInventario()
{
    char respuesta;

    cout << "\nEsta seguro de reiniciar inventario? (S/N): ";
    cin >> respuesta;

    if(respuesta != 'S' && respuesta != 's')
    {
        cout << "\nOperacion cancelada.\n";
        return;
    }

    fstream archivo("productos.dat", ios::binary | ios::in | ios::out);

    Producto p;

    while(archivo.read((char*)&p, sizeof(Producto)))
    {
        p.stock = 0;
        p.vendidos = 0;

        archivo.seekp(-sizeof(Producto), ios::cur);
        archivo.write((char*)&p, sizeof(Producto));

        archivo.seekg(archivo.tellp());
    }

    archivo.close();

    cout << "\nInventario reiniciado correctamente.\n";
}

void estadisticasGenerales()
{
    Producto p;

    ifstream archivo("productos.dat", ios::binary);

    int totalProductos = 0;
    int stockTotal = 0;
    int vendidosTotal = 0;

    double valorInventario = 0;

    while(archivo.read((char*)&p, sizeof(Producto)))
    {
        if(p.activo)
        {
            totalProductos++;

            stockTotal += p.stock;

            vendidosTotal += p.vendidos;

            valorInventario += (p.stock * p.precio);
        }
    }

    archivo.close();

    Venta v;

    ifstream ventas("ventas.dat", ios::binary);

    int totalVentas = 0;

    double dineroGenerado = 0;

    while(ventas.read((char*)&v, sizeof(Venta)))
    {
        totalVentas++;

        dineroGenerado += v.total;
    }

    ventas.close();

    cout << "\n===== ESTADISTICAS GENERALES =====\n";

    cout << "\nProductos activos: "
        << totalProductos;

    cout << "\nStock total: "

```

```
<< stockTotal;

cout << "\nProductos vendidos: "
<< vendidosTotal;

cout << "\nVentas registradas: "
<< totalVentas;

cout << "\nDinero generado: Q"
<< fixed
<< setprecision(2)
<< dineroGenerado;

cout << "\nValor del inventario: Q"
<< fixed
<< setprecision(2)
<< valorInventario
<< endl;
```